SAMVAADH Podcast - Conversations You Can Take Anywhere

Anantha Krishna Chilappagari
Department of Information Science
University of North Texas
Denton, Texas, United States
ID: 11670693

Nagarjuna Pendekanti
Department of Information Science
University of North Texas
Denton, Texas, United States
ID: 11711371

Snigdha Reddy Nannuri
Department of Information Science
University of North Texas
Denton, Texas, United States
ID: 11688316

Krishna Charan Kandlakunta
Department of Information Science
University of North Texas
Denton, Texas, United States
ID: 11669902

Ravi Kumar Bevara
Department of Information Science
University of North Texas
Texas, United States
Ravikumar.bevara@my.unt.edu

Stephen Wheeler
Department of Information Science
University of North Texas
Texas, United States
stephen.wheeler@my.unt.edu

***Abstract*—The SAMVAADH Podcast is an AI-based System for Conversational Podcasting. SAMVAADH Podcast is a software solution that automatically converts academic papers into conversational audio podcasts in a language suitable for major audiences like students, scholars and researchers. It tackles that challenge of keeping up with the research by turning arXiv papers into digestible conversational podcasts on various research and academic papers and articles. The system uses a robust pipeline of length-controlled text generation, including a GPT-4 mini finetuned model on a huge database of Paper2Podcast dataset consists of academic podcast transcripts and length control, including careful prompt generation to target desired duration. The created script is subsequently transformed into speech with an OpenAI text-to-speech model, yielding a lively audio podcast.**

I. Introduction and Statement of the Problem

In recent years, the volume of content and data in general, has been growing exponentially and in this age of Artificial Intelligence there are many sources to consume the data staring from a 5 second shorts to 2 to 3 hours of podcast everything is a piece of content and is being consumed by the people across the globe. Similarly like any other data the number of research papers, articles, Academic papers had grown at an unprecedented rate.

There are thousands of publications who are publishing such research articles and papers each month rapidly across variety of fields including Computer Science, Physics, Mathematics, and Quantitative biology, Quantum Computing making it mammoth task for researchers, professionals, and students to keep pace with the latest discoveries.

Traditional methods of consuming the research, articles involve the long documents requires copious time and concentrated attention, and technical jargon and complex terminology can make it difficult for non-experts or interdisciplinary audiences to understand. Technology and knowledge should be accessibility should not be restricted to scholarly personals but also to budding students and beginners who has acute ability to understand the complex topics.

At the same time, listening to podcasts has become increasingly popular and Podcasts are now a common informal learning tool that allow listeners to absorb knowledge while multitasking, working out, or traveling. There is a significant difference between these two trends although podcasts democratize knowledge across a wide range of fields, there is currently no automated, scalable method for converting the expanding body of scholarly literature into excellent, dialogue-style audio presentations is our main objective for *SAMVAADH Podcast – Conversations You Can Take Anywhere*

In order to fill this gap, we have chosen to develop the AI-generated podcast "SAMVAADH Podcast – Conversations You Can Take Anywhere," which offers a comprehensive, end-to-end framework for turning academic papers into polished, multi-speaker podcast episodes.

II. Review of Literature

Several strands of research and development intersect in our project, including AI-generated podcasts, controllable text generation, and educational podcast design. We review key literature below, highlighting each work's contributions and how they inform or contrast with SAMVAADH.

Yahagi et al. [1] introduced PaperWave, a proof-of-concept system for transforming academic papers into chatty podcasts with the help of LLMs. PaperWave showed that large language models could write dialogue that communicated what a paper was about in an easier to understand way. more appealing presentation than a monologue. SAMVAADH is an extension of addons and builds upon the concept of- utility to complete the task in seconds.PaperWave gets turned into a podcast. a more sophisticated GPT-4-level model and explicitly including length control. In that respect, PaperWave stands in contrast with (17),where proved the overall idea, our system makes certain the podcast occurs in a fixed amount of time (5 off, 10

off, 15 off, etc) a to be an important require- ment for practical applications, but it was not well-treated in [1].

Liang *et al.* [2] give an extensive overview of controllable text generation for large language models. They classify approaches as model retraining, fine-tuning, reinforcement learning, prompt engineering, and decoding-time methods. The survey points out that, in real-world applications, outputs have to satisfy some constraints (such as length, style, content, and so on). SAMVAADH uses a few of the observations from this survey. In particular we leverage fine-tuning to make the model conversational in nature (an *attribute control* modality from [2]) and prompt engineering for turning off/on output length and formatting.

Dobozy, [3] also examined the effects of dialogue design and voice in educational podcasts. While our approach predates more recent advances in AI, we use this as a motivation for our conversational presentation. Dobozy discovered that *interactive lecture podcasting* can enhance learner engagement and even Students participation.

Gu *et al.* [4] focus on length-controlled text generation for black-box LLMs. They design an iterative sampling scheme via Metropolis-Hastings (MH) algorithm to direct the LLM's output toward a target length while keeping internal parameters unchanged. This is also very interesting in the context of SAMVAADH since one of our goals is to produce podcast scripts of certain duration at the user end wishes. We further take up the notion of [4] and post-process the script by an MH-like sampling procedure.

Jie *et al.* [5] propose a prompt-driven approach to length-controlled generation, which achieves high accuracy of target-length hitting across different control settings. Their text summarization procedure, specifically, leverages well-engineered prompts and reinforcement learning fine-tuning to generate outputs of the correct length. SAMVAADH is motivated by [5] and employs explicit prompt-based instructions for length control (for example, it requires the words in the prompt to explain the length of a 10-minute podcast). While Jie *et al.* use their approach in the news and summarization domain, and we generalize the idea to a new domain of automatically generating podcast scripts from research papers. Our system prompt contains both the length prompts and the speaker acting as practiced in e.g. the setting "two speakers" etc., therefore effectively combining both length and style as per the multi-control approach to dialogue in [5]. A limitation of [5] is the trade-off between length control and content richness, which SAMVAADH overcomes by fine-tuning on the real podcast data.

Pojoni *et al.* [6] investigated the application of GPT-4 in argument mining over podcast data, and demonstrated that LLMs can process and retrieve structured arguments from conversations in the form of textual transcripts. This work is indirectly related to ours: [6] does not create podcasts as we do, but rather employs AI to analyse them. But it highlights the degree to which GPT-4 will offer insight into conversational input. SAMVAADH takes advantage of this observation—using an LLM not only to generate fluent dialogue but also to organizes dialogue in a way that salient claims and conclusions of the paper are introduced by the "hosts" in a natural way. That is, when Pojoni *et al.* apply GPT-4 to identify arguments in existing dialogue, and similarly we apply a GPT-4-based method to generate dialogue containing the crucial arguments.

Kokala [7] deals with the incorporation of Google's NotebookLM (a large language model based research assistant) into content creation for personalisation of the podcast generation. This paper demonstrates that AI has the power to optimize podcast scripting and customize content to the listener. The challenging aspects in [7] are dynamic topic picking, personalized report based on user profile and automatic editing of the output. SAMVAADH has the same high-level goal as bringing new technology to bear on the problem of podcast show production – but our vision is narrower and goes deeper into academic research. Unlike the broader context of personalized content in Kokala (e.g., adapting to user application or user's interest or knowledge level), our approach focuses for the moment on faithful rendering of the source paper and on summarization with time constraints. We do not apply the user-level personalization apart from length choice as in [7]. Nonetheless, our multi-agent framework.

Khanna *et al.* [8] present *SummarizeAI*, a system to produce both text summaries and audio abstracts of podcasts. Basically, [8] solves our inverse problem: given a long podcast, output a long textual summary and optionally a short audio. The pipeline that SummarizeAI uses includes an NLP algorithm to generate the summary and a TTS algorithm to speak the summary, and is similar in idea to what SAMVAADH has as its pipeline (input is text and our output is a new episode). The success from [8] in using large language models as the summarization backbone supports the observation that LLMs can be an effective instrument to compress spoken content. We extrapolate this to written text and use an LLM to compress and condense an explanation of a research paper. One limitation of SummarizeAI is that it may not be equipped to deal with technical text beyond average podcast topics, while SAMVAADH deals with academic papers and is further fine-tuned for such content. In matters of idea adoption, we are inspired by the concept of interleaving LLM based content compression with TTS as in [8], but whereas it lacks an interactive presentation and flexibility around the desired length, we incorporate a user-guided length as they appear in SummarizeAI. .

III. Study Objectives

The project goals are as follows:

- **Automated Podcast Creation** Create a system that can automatically generate a conversational (podcast-like) scripted conversation on a 'pre-recorded' research paper (as input) with minimal human assistance.
- **Length Control**: Support for user-specified content durations (i.e., number of minutes), so the generated script respects the time-limit specified by broadcasters (e.g., make it a 5-minute brief, or a 30-minutedeep dive).

- **Conversational Sincerity**: Generate dialogues that describe the science in plain and clear language (like an expert explaining to a layperson) without obfuscating important technical details to elevate listener comprehension.
- **Model Fine-tuning:** We fine-tune a large language model on a corpus of podcast transcripts (Paper2Podcast dataset) consists of 1063 transcripts such that the model learns the structure and style of research discussions, which in turn leads to more natural and coherent generated content.
- **5 End-to-End Deployment:** Combine the components (paper retrieval, text processing, script generation, text-to-speech) into a complete application application with a user-friendly interface to show the viability of the approach for real-world application.
- **Quality Measurement**: Evaluate the quality of the generated podcasts with automatic metrics (with respect to reference summaries or transcripts) and qualitative assessments, to pave the way for future refinement and user studies.

IV. Data Collection

To support this project, data was gathered from two primary sources: research papers themselves and a dataset of existing podcast transcripts.

First, we used the **arXiv API** to obtain publications and their metadata. The arXiv API lets you programmatically search for and access arXiv papers by title, author, abstract keywords, or arXiv ID. For the purposes of our system, it provides the API for which users can search for a paper which they are interested in (e.g. via keywords or a known arXiv id). If a paper is chosen, then the system automatically downloads the paper's PDF. The full-body of the paper is then output as input to the podcast generation script. This approach ensures that our system can access up-to-date research content on demand, rather than relying on a fixed corpus of papers. We did not keep the entire arXiv corpus in a local repository, with papers being fetched on demand it is scalable and always up-to-date.

Secondly, we compiled a custom **Paper2Podcast dataset** available at https://www.paper2podcast.com that has around 1000 transcripts in the domain of podcast episodes about research papers. These transcripts were assembled from educational and research oriented podcasts that discuss research articles that made by transcribing in form of podcasts text. For each data sample in the dataset, we have the reference to the original paper, as well as the dialogue itself. We have scraped the website and compiled 100 research papers and transcripts. We used this corpus to fine-tune our language model so that it learns the nuances of academic discussion in podcast form. based on this Our model helps to generate podcast with specified length asked by the user.

V. Exploratory Data Analysis and Hypotheses

Upon examining numerous transcripts, we observed a fairly consistent **structure in podcast conversations** about research papers. Typically, an episode would start with a brief introduction where the hosts greet each other and introduce the paper or topic of discussion. This is often followed by a background segment, where one host might ask, "What is this paper about and why is it important?" and the other provides context or motivation. The bulk of the conversation then delves into the methodology and findings: one speaker might explain the approach while the other interjects with clarifying questions or analogies (emulating a curious audience's perspective).

Linguistically, the transcripts showed that the language used is less formal than the source papers. Jargon is often explained or replaced with simpler terms, and personal pronouns ("we", "you") appear frequently as the hosts engage in dialogue. There are also discourse markers like "So," "Let's dive into…", or "That means…", which make the conversation flow naturally. We identified these as important stylistic elements to emulate. Based on this analysis, we formulated the following hypotheses for our project:

- **H1: Fine-tuned model yields better structure and coherence.** A language model fine-tuned on podcast transcripts will more reliably produce the desired dialogue structure (introduction, explanation, conclusion) and maintain a conversational tone than a model that is only prompted but not fine-tuned. We expect the fine-tuned model to insert the kinds of transitions and clarifications we saw in the dataset, leading to a script that feels more like a real podcast.
- **H2: No major loss in content from length compression.** When instructing the model to generate, say, a 5-minute vs. a 15-minute podcast from the same paper, we anticipate that the shorter one will cover the core findings but omit detailed examples or technical nuances. Our hypothesis is that by training on diverse transcript lengths, the model will prioritize essential information for shorter outputs and naturally include more detail for longer ones. Thus, even under length constraints, the vital points of the paper will consistently appear in the script.

These hypotheses guided the design of our generation approach. For instance, H1 and H4 justified the effort to fine-tune on real dialogues rather than relying on generic GPT-4 output, and H2 informed our decision to implement a length-control algorithm on top of prompting. As we will discuss in the Results section, we devised experiments to test these hypotheses, such as comparing outputs from fine-tuned vs. base models and with vs. without the MH controller.

In summary, our exploratory data analysis of existing podcasts reinforced the intuition that conversational format and content tailoring (in both style and length) are key to effective audio summaries of papers. The hypotheses derived from this analysis set the expectations for the performance of SAMVAADH and shape how we evaluate its success.

VI. Data Analytics (Methodology)

SAMVAADH system consists of modules inter-dependent upon each other, trained to work in sequence mode of a chain of steps, mimicking the process flow from paper input to

output in podcasts. Methodology and technicalities involved therein are given below: Data flow in the system, use of prompt engineering and fine-tuning procedures, and memory and length control system integration.

A. System Architecture Overview

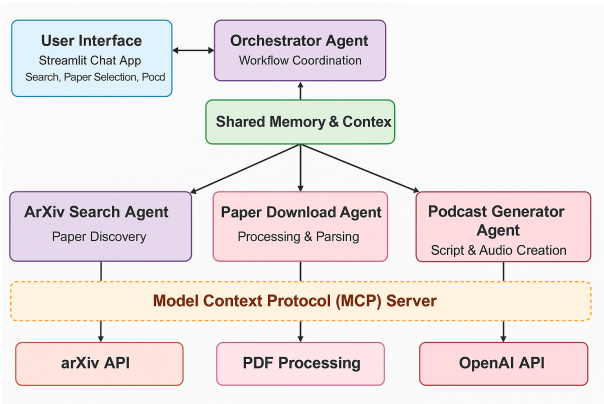

Fig. 1. Architecture

1) **Paper Search**: A *Search Agent* calls the arXiv API to get candidate papers corresponding to the user's input (title substring or keywords). The agent displays a set of candidate papers (title, authors, abstract). The user selects the target paper from among them.
2) **Academic Paper Download and Parsing**: An example test case of one of the chosen test papers' *Download and Parse Agent* is employed. The agent downloads the PDF of the paper (from the arXiv ID) and obtains text content of the paper. Text filtering for recognizing important parts like "Introduction," "Methods," "Results," and "Conclusion" are employed. We employed a basic parser to extract the section headings from paper text, then divided the text into sections on the basis of them. This gives us an uncluttered, orderly presentation of paper contents in dictionary-like manner with section headings referring to respective content, and such content retained in system context.
3) **Podcast Script Generation**: Having now parsed, the system also has a clue about how it is supposed to be read. The user can be requested to input a target duration of a podcast (otherwise). Default time slots include 5, 10, 15, 20, or 30 minutes. It is a trained GPT model invoked by a *Podcast Generator Agent* to create the script. His/her task is to compose a script with dialogue adequate to answer most of the questions required by the paper.
   Audio Synthesis: Finally, an *Audio Agent* reads the audio file synthesis of the script. It is performed through OpenAI's TTS API and pre-trained voice models for synthesizing the dialogue as an audio file. It is played back to the user in the GUI as an audio player and the text transcript finally.

To handle these tasks collectively, we have developed an **Orchestrator** that tries to orchestrate the various agents. The orchestrator facilitates explicit handover: i.e., after the search results are displayed and the user has selected a paper, the orchestrator invokes the download/parse agent independently; after it finishes, it invokes the generation agent independently, etc. This keeps the end-user experience simple, and usage is as simple as entering a question in and then listening to the podcast later.

B. AppContext and Memory Management

State was divided between multiple agents and UI operations within an AppContext object. AppContext holds the following:

- The arXiv ID or metadata of the chosen paper.
- Full text and structure parsed for the paper.
- User's preferred podcast length and options (e.g., speakers if that was an option that could be turned on/off).

By keeping this information within a shared context, every agent can retrieve what it needs without requiring repeated input from the user or re-calculation. For instance, the Podcast Generator Agent can simply retrieve the parsed sections from AppContext rather than re-reading the PDF directly.

Apart from that, we employed **LangChain's ConversationBufferMemory** to save the evolving conversation between the system and user. The memory comes in handy in the event that the user may provide follow-up questions or require changes. For example, if the user has listened to the podcast, they may reply with, "Can you explain the methodology?" In this case, our system (via LangChain memory) recalls what paper and what material was under discussion so that a helpful response can be generated. We used the memory to store the latest N turns of conversation, and we also had a backup to store all the conversations in case the primary memory becomes full or gets reset.

The use of AppContext and LangChain memory both supplies *persistent knowledge* regarding the document and *conversational continuity*. The structure is modular: paper and result content-state-dependent management by AppContext, and dialogue history management by memory. This was helpful when we were writing and debugging since we were able to examine the context object and see what the system "knows" at a given time. while the memory handles dialogue history. This proved helpful during development and debugging, as we could inspect the context object to see what the system "knows" at any point.

C. Prompt Engineering for Script Generation

One of the most important aspects of our solution is prompt engineering of the podcast script generation. Through fine-tuning the model, we found that providing a well-structured, formatted prompt at runtime greatly improves the consistency of the outputs, specifically for adhering to meeting length requirements and following the right format.

The system produces a prompt that has two parts: a *system message* that specifies the general behavior, and a *user message* that contains the task-specific content and instructions. The system prompt includes context such as: "You are an AI assistant that generates a discussion between podcast hosts regarding a research paper. Your tone is explanatory, engaging, and to the point. You have the following resources: knowledge of the paper's text and an intended discussion length." It may also include instructions such as "Avoid technical jargon without explanation" and stylistic requests such as "Use the Feynman technique to explain complex concepts."

Our user prompt then simply says something along the following lines: "Prepare an [X]-minute podcast script of approximately [Y] words (with a margin of $\pm$Z words) based on the following research paper. The script must be in the form of a discussion between 2 hosts, a male and a female, named Alice and Bob (for example). The content of the paper is presented below. Highlight major findings and make sure that the discussion logically progresses from introduction to conclusion." We insert the actual content of the paper after this prompt, typically not the whole text but the major points. Practically, we provide either the abstract and selected excerpts of each section, or, if the model context window allows, a truncated full text (we limited to e.g. 150k characters to ensure it will fit in token limits with room for output).

The prompt also includes a target word count corresponding to the target duration (e.g., 10 minutes $\approx$ 1600 words, assuming an average speech rate of 160 words per minute). We discovered that having the word count explicitly declared (with some leeway, e.g., $\pm$100 words) in the instruction tends to cause the model to concentrate and self-censor the output length. This was inspired by Jie *et al.*'s [5] prompt-based control method. For instance, the model may receive an instruction: "Generate a script of 1600$\pm$100 words."

D. Metropolis-Hastings Length Control

Even with strong prompt engineering, the models at times exceeded the length limit, especially if the subject matter is extremely sensitive or the model goes hyperactive to generate content on the subject. For this, we utilized an optional **Metropolis-Hastings (MH) based length control mechanism** by Gu *et al.* [4]'s approach.

In our approach, after we have developed an initial script, we check that it meets the target length. If its length is close to the target range (i.e., within plus or minus 5% of the target length), then the length conditions are met. Otherwise, we invoke the MH-based process:

1) We take the developed script as an initial solution in a candidate script space.
2) Next, we randomly choose a section of the script to remove or add. If, for example, the script is too brief, we instruct the model to "add more details on X section". If the script is long, we find out where the script will tend to be wordy or include redundant facts and tell the model to make it shorter or cut them out in a new script.
3) The model generates a altered script (proposal state). We calculate the similarity of the novel script to the target. If it is nearer to the target than the present state, we accept the alteration..
4) This process is repeated for a fixed number of iterations or until the length is within the proper range.

E. Model Details and Fine-Tuning

The base model employed in SAMVAADH is called *GPT-4o mini*. In practice, We fine-tuned on the Paper2Podcast dataset and had the model take a paper's abstract (and highlights) as input and output a conversation. This was trained on a cloud GPU device, training our 1000 sample transcripts for a few epochs. We tuned relatively the temperature parameter in an effort to have some flexibility of use of language but not to engage in non-factual responses. The fine-tuning was good at initial testing. The fine-tuned model's output was smoother and less full of distracting tangents than from the same inputs with the baseline GPT-3.5 model. The dialogue sounded more "trained" to read like a podcast, probably because the model was trained on common styles in the transcripts (like the question-and-answer style of questioning).

F. Streamlit App and Multi-Agent Interface

We developed the front-end of SAMVAADH using Streamlit with which we could seamlessly create an interactive web interface. The multi-agent system is run in this app, and we used an event-driven approach for updating the UI after every step. For example:

- After finishing the search step, a list of results is displayed with select buttons.
- Upon being parsed and downloaded, the user interface can display a message "Paper downloaded. Detected sections: Introduction, Methods, Results, Conclusion…" prompting for a time.
- When generating, a bar or status message "Generating podcast script…" will show.
- Lastly, there is an available audio player widget with the transcript text below, to be read or saved for use by the user.

VII. Data Visualization and Results Report

We present two core result tables comparing the base and fine-tuned models: automatic evaluation metrics (BLEU-4, ROUGE-L F1) and human evaluation scores (length accuracy, technicality, relevance, explainability).

A. Automatic Evaluation Metrics

Table I reports BLEU-4 and ROUGE-L F1 scores for each model.

B. Human Evaluation

Table II shows averaged human ratings (5-point Likert) and length-accuracy percentages.

TABLE I
AUTOMATIC EVALUATION METRICS FOR BASE AND FINE-TUNED MODELS

Model	BLEU-4	ROUGE-L F1
Base	0.40	0.45
Fine-tuned	0.48	0.50

TABLE II
HUMAN EVALUATION SCORES FOR BASE AND FINE-TUNED MODELS

Model Type	Length Acc.	Tech.	Relev.	Explain.
Base Model	65%	3.20	3.50	3.50
Fine-tuned	70%	3.91	3.97	3.70

- The fine-tuned model achieves 70%, outperforming the base model's 65%.
- Technicality: Fine-tuned (3.91) vs. Base (3.20), showing a strong boost in subject-matter depth.
- Relevance: Fine-tuned (3.97) vs. Base (3.50), indicating more on-target responses.
- Explainability: Fine-tuned (3.70) vs. Base (3.50), reflecting clearer, more understandable outputs.

VIII. PODCAST GENERATION RESULTS

Figure 2 shows a screenshot of our ArXiv Podcast Generator in action. It demonstrates the full pipeline from user query to audio output:

Fig. 2. Output of the ArXiv Podcast Generator

As illustrated, the user types a natural-language query into the chat interface and the back end:

1) Searches arXiv for matching papers (here, "Attention Is All You Need").
2) Displays the paper's title, full author list, arXiv identifier, and estimated podcast duration.
3) Generates and saves a narrated podcast audio file (`output/podcast.mp3`).
4) Embeds a standard HTML5 audio player for immediate in-browser listening.
5) Offers a direct "Download Podcast" button for offline access.

This end-to-end demonstration confirms that our system can convert any arXiv paper into a concise, user-friendly audio podcast with natural language of the user.

IX. LIMITATIONS

The SAMVAADH system demonstrates effectiveness in creating conversational podcast episodes from research papers yet faces multiple limitations. The system fails to generate text outputs longer than 3,000 words or 20 minutes through its API interface. The current pipeline extends to this threshold but creating longer episodes remains difficult and unpredictable. The length control methods Metropolis-Hastings and prompt-based techniques work for short durations but fail to manage text generation when exceeding the 3,000-word limit thus limiting scalability for extended podcasts. The system faces limitations from API token restrictions which prevent it from processing research papers containing extensive contexts. The system's ability to process complex academic documents and generate complete summaries and dialogues is restricted because of this limitation

X. CONCLUSION

This project produced **SAMVAADH: Conversations You Can Take Anywhere**, a system for converting research papers into conversational podcast episodes through LLMs. The project applied modern AI-based summarization together with dialogue creation techniques and text generation control methods to create audio-friendly academic content. The proposed pipeline system that combined search functionality with MarkItDown MCP document conversion and section-by-section LLM summarization and TTS proved useful for converting complex papers into meaningful dialogues. The experimental findings validate the initial prediction that large language models serve as efficient tools for generating both informative and engaging spoken summaries from complicated texts. The quantitative analysis revealed strong similarity between our podcast outputs and reference summaries (ROUGE-L and BLEU Score) and listeners found the conversational style effective for understanding the information.

We validated our system by demonstrating our prediction that podcasts can expand the accessibility of academic content for users who want to learn about research breakthroughs through AI-created conversations which they can bring with them in any location. Our research demonstrates that LLM context limits and TTS expressiveness are resolvable with present methods but provide well-defined development objectives for future advancement.

REFERENCES

[1] Y. Yahagi *et al.*, "PaperWave: Listening to Research Papers as Conversational Podcasts Scripted by LLM," *arXiv preprint arXiv:2410.15023*, 2024.
[2] X. Liang *et al.*, "Controllable Text Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2408.12599*, 2024.
[3] E. Dobozy, "Interactive lecture podcasting: Probing the impact of dialogue design in LAMS," in *Proc. Int. LAMS Conf.*, 2009.
[4] Y. Gu *et al.*, "Length controlled generation for black-box LLMs," *arXiv preprint arXiv:2412.14656*, 2024.

[5] J. Jie *et al.*, "Prompt-based length controlled generation with multiple control types," in *Proc. ACL Findings*, 2024.

[6] M.-L. Pojoni, L. Dumani, and R. Schenkel, "Argument-mining from podcasts using ChatGPT," in *Proc. ICCBR Workshops*, 2023, pp. 129–144.

[7] A. Kokala, "Revolutionizing content creation: leveraging AI-driven podcast generation with NotebookLM and personalized insights," *Int. Res. J. Modernization Eng. Tech. Sci.*, vol. 6, no. 12, pp. 1798–1804, 2024.

[8] D. Khanna, R. Bhushan, K. Goel, and S. Juneja, "SummarizeAI – Summarization of the podcasts," SSRN Preprint 4628657, Nov. 2023.

[9] A. Ghazimatin, E. Garmash, G. Penha, K. Sheets, M. Achenbach, O. Semerci, R. Galvez, M. Tannenberg, S. Mantravadi, and D. Narayanan, "PODTILE: Facilitating Podcast Episode Browsing with Auto-generated Chapters," in Proc. 33rd ACM Int. Conf. on Information and Knowledge Management, pp. 4487–4495, 2024.

[10] Y. Xiao, L. He, H. Guo, F. Xie, and T. Lee, "PodAgent: A Comprehensive Framework for Podcast Generation," arXiv preprint arXiv:2503.00455, Mar.2025.